Implicit pangenome graphs

Andrea Guarracino[1,2*], Bryce Kille[3], ... and Erik Garrison[1*]

[1]Department of Genetics, Genomics and Informatics, University of Tennessee Health Science Center, Memphis, TN 38163, USA. [2]Bioinnovation and Genome Sciences, The Translational Genomics Research Institute (TGen), Phoenix, AZ 85004, USA. [3]Illumina Inc., San Diego, CA 92122, USA.

*Corresponding author. E-mail: aguarracino@tgen.org; egarris5@uthsc.edu;

Abstract

Querying genomic relationships across hundreds of genomes currently requires either constructing computationally expensive graph structures or using projection tools limited to pairwise mappings between individual genomes. Here we introduce implicit pangenome graphs, a framework that treats pairwise alignments as a queryable pangenome representation, making population-scale analysis accessible on commodity hardware without upfront graph construction.

Keywords: pangenomes, pangenome graphs, genome alignment, transitive queries, coordinate projection, population genomics

Whole-genome alignments are a foundational data layer for pangenomics [1–3], yet current workflows treat them as intermediates consumed during graph construction [4, 5]. We propose that the alignment collection itself—together with the transitive relationships it encodes—constitutes a directly queryable pangenome representation. Operating in this alignment space, we can project coordinates transitively across genomes, partition the pangenome along alignment connectivity, and materialize explicit graphs only when needed for specific regions of interest. As high-quality assemblies are now routinely produced for hundreds to thousands of individuals [6, 7], this alignment-first approach avoids the substantial computation required for monolithic graph construction at population scale.

Pairwise alignments inherently define a variation graph: positions matched transitively across alignments correspond to shared nodes in a graph that losslessly encodes both the sequences and their relationships [8]. The pangenome graph is thus implied by the alignments, even when it is never constructed. Graph construction methods—seqwish [8], Cactus [9], TBA/MULTIZ [10]—exploit this property to build pangenome graphs and multiple alignments, but require materializing the complete structure before any region can be examined. Coordinate projection tools such as liftOver [11] and paftools [12] operate at query time but map positions between only two genomes; halLiftover [13]

achieves multi-hop projection but requires a pre-computed hierarchical alignment. No existing tool exploits this transitive structure at query time directly from pairwise alignments, without first constructing a graph or hierarchical alignment.

IMPG (Implicit Pangenome Graph) operates directly on pairwise alignments as implicit graph structures (Figure 1A), providing three core capabilities: (a) transitive coordinate projection across all genomes in a pangenome, (b) alignment-based partitioning for distributed graph construction, and (c) on-demand graph materialization for regions of interest. Rather than materializing the full implicit graph, IMPG indexes alignment files using cache-oblivious interval trees [14] and traverses them at query time to discover both direct and transitive relationships. Given coordinates on any genome, IMPG projects them onto all other genomes, functioning as a pangenome-wide coordinate liftover. IMPG supports alignments as PAF files (uncompressed and BGZF-compressed), .1aln [15], and TPA [16]—the latter two based on compact tracepoint representations of the alignments—and sequences as FASTA (uncompressed and BGZF-compressed) or AGC archives [17]. This flexibility allows integration into existing workflows; supporting tracepoint-based formats and compressed genome archives enables population-scale queries on a single workstation. On 466 human haplotypes aligned to CHM13, IMPG indexes 7.1 million alignments in 10–18 seconds and queries a 6 Mbp region in under 2 seconds, with peak memory of 40–88 MiB (Supplementary Table 1). Transitive queries from a non-reference haplotype reach all 466 assemblies through multi-hop traversal—from 2 haplotypes (direct) to 466 (transitive)—demonstrating that the implicit index captures pangenome-wide connectivity (Extended Data Figure 2). The complete ready-to-query representation of 466 haplotypes—per-file TPA alignments, bidirectional indexes, and a compressed sequence archive—totals 42.5 GiB (Supplementary Table 1). The entire human pangenome fits on a laptop disk and can be queried in seconds with minimal memory; no decompression or graph loading step is required.

As pangenome datasets grow to hundreds or thousands of haplotypes, monolithic graph construction becomes intractable: memory, computation, and time scale super-linearly with the number of sequences. Partitioning is essential, but existing approaches operate at whole-sequence granularity. Chromosome-based partitioning assigns entire contigs by chromosome identity; community-based approaches such as PGGB's Leiden clustering [4] assign entire contigs to communities based on sequence-similarity networks. Neither can split a contig: if one region of a contig is homologous to chromosome 1 and another to chromosome 5, the entire contig is forced into one partition.

IMPG introduces alignment-based partitioning, which operates at aligned-interval granularity (Figure 1B). The `partition` command uses sliding windows of transitive queries to divide the pangenome into segments sharing alignment connectivity, and critically, it can split a single contig across multiple partitions at alignment boundaries. This naturally handles cases

where different regions of a contig belong to different genomic contexts—including translocations, inter-chromosomal rearrangements, and fragmented assemblies—without requiring special-casing or forcing the entire contig into one partition.

This partitioning splits the pangenome into manageable chunks for any downstream analysis—similarity computation, sequence extraction, genotyping, or graph construction. For graph construction, each partition is processed independently by any tool—PGGB, vg [18], or custom pipelines—distributing computation and memory across nodes; the `lace` command then merges the resulting subgraphs into a unified GFA or VCF. When a researcher needs to study region-specific variation—visualizing a complex structural variant, inspecting bubble structures, or calling variants against local topology—IMPG can materialize graphs on demand via three built-in engines: seqwish-based graph induction [8, 19], POA via SPOA [20], and a recursive divide-and-conquer engine that chunks homologous sequences, builds POA sub-graphs in parallel, and laces them into a single variation graph. The `align` command supports this by generating sparsified alignment sets guided by Erdős–Rényi connectivity thresholds [21].

IMPG is already production infrastructure for structural genotyping. The COSIGT pipeline [22] uses `impg query` to project locus coordinates onto all haplotypes and `impg refine` to optimize locus boundaries, then constructs local pangenome graphs for genotyping, demonstrating that alignment-space operations can underpin population-scale genotyping of complex variation.

To illustrate IMPG's capacity for rapid population-scale exploration, we investigated a tandem repeat cluster at chromosome 11q23 that Nishikawa et al. [23] identified as bound by the Epstein–Barr virus EBNA1 protein, inducing chromosomal fragile site breakage. The original study examined four long-read genomes—two Ashkenazi and two Chinese—and reported a ∼2.6-fold difference in repeat copy number between the two groups. Using IMPG, we queried this ∼21 kb locus across all 466 HPRCv2 haplotypes in seconds, revealing that repeat region lengths vary continuously across diverse human populations and extend well beyond the range observed in four genomes (Figure 2). The four-genome observation was an undersampled snapshot: population-scale querying shows that this polymorphism is far more complex than a simple two-group difference, underscoring how small sample sizes can mischaracterize structural variation at clinically relevant loci.

More broadly, implicit pangenome graphs introduce a new layer in the pangenome analysis stack. Rather than requiring a single monolithic construction step, the alignment-space framework lets users query, partition, and extract regions on demand, then selectively invoke graph construction only where explicit topology is needed—for bubble enumeration, snarl decomposition, or graph-aware variant calling via tools like Minigraph-Cactus [5] or personalized pangenome references [24]. IMPG can supply these builders with partitioned input through its partition-lace pipeline, or materialize

graphs on demand for specific loci. The two representations are complementary: alignment space provides the upstream infrastructure for exploration and partitioning, while graph space provides the downstream topology for variant-aware analyses.

IMPG's projection accuracy is determined by the quality of the input alignments: given correct alignments, exact-mode queries—which traverse full CIGAR strings (PAF) or reconstruct them via wavefront alignment from tracepoints (.1aln/.tpa)—produce base-exact coordinates by construction. For tracepoint-based formats, an approximate query mode bypasses CIGAR reconstruction entirely, estimating projected boundaries from tracepoint segment coordinates; this is faster but boundaries may differ from exact projections by up to the tracepoint segment size per hop (e.g., $\leq$100 bp for fixed-spacing tracepoints; Supplementary Table X). Regions absent from input alignments cannot be recovered. The incremental design means each new assembly requires alignment against only a spanning subset of existing haplotypes, not a complete rebuild—enabling continuous growth of the pangenome index. Looking forward, transitive queries yield pairwise similarity across indexed haplotypes, opening the possibility of computing population genetics summary statistics directly from alignment space without decomposing variation into biallelic sites. By operating in alignment space, IMPG makes it possible to study sequence variation across populations at scale without requiring upfront graph construction—bridging the gap between assembly production and population-level genomic analysis.

Acknowledgments

We thank the Human Pangenome Reference Consortium for making the HPRCv2 assemblies publicly available. We thank Max Marin for the EBV repeat locus analysis.

Author Contributions Statement

A.G. developed the software, performed benchmarking and analysis, created visualizations, and wrote the manuscript. B.K. contributed to software development. E.G. conceived the project, developed the software, acquired funding, and wrote the manuscript.

Competing Interests Statement

B.K. is an employee of Illumina. Other authors declare no competing interests.

Funding

A.G. and E.G. are funded by NSF PPoSS Award #2118709; NIH R01HG013618; NIH U01HG013760; NIH U01DA057530; NIH U41HG010972; and the Tennessee Center for Integrative and Translational Genomics.

References

[1] Consortium, C.P.-G.: Computational pan-genomics: status, promises and challenges. Briefings in Bioinformatics **19**(1), 118–135 (2018)

[2] Eizenga, J.M., Novak, A.M., Sibbesen, J.A., Heumos, S., Ghaffaari, A., Hickey, G., Chang, X., Seaman, J.D., Rounthwaite, R., Ebler, J., Rautiainen, M., Garg, S., Paten, B., Marschall, T., Sirén, J., Garrison, E.: Pangenome graphs. Annual Review of Genomics and Human Genetics **21**(1), 139–162 (2020)

[3] Paten, B., Novak, A.M., Eizenga, J.M., Garrison, E.: Genome graphs and the evolution of genome inference. Genome Research **27**(5), 665–676 (2017)

[4] Garrison, E., Guarracino, A., Heumos, S., *et al.*: Building pangenome graphs. Nature Methods **21**(11), 2008–2012 (2024)

[5] Hickey, G., Monlong, J., Ebler, J., *et al.*: Pangenome graph construction from genome alignments with Minigraph-Cactus. Nature Biotechnology **42**(4), 663–673 (2023)

[6] Liao, W.-W., Asri, M., Ebler, J., *et al.*: A draft human pangenome reference. Nature **617**(7960), 312–324 (2023)

[7] Wang, T., Antonacci-Fulton, L., Howe, K., *et al.*: The human pangenome project: a global resource to map genomic diversity. Nature **604**(7906), 437–446 (2022)

[8] Garrison, E., Guarracino, A.: Unbiased pangenome graphs. Bioinformatics **39**(1), 743 (2023)

[9] Paten, B., Earl, D., Nguyen, N., Diekhans, M., Zerbino, D., Haussler, D.: Cactus: Algorithms for genome multiple sequence alignment. Genome Research **21**(9), 1512–1528 (2011)

[10] Blanchette, M., Kent, W.J., Riemer, C., Elnitski, L., Smit, A.F.A., Roskin, K.M., Baertsch, R., Rosenbloom, K., Clawson, H., Green, E.D., Haussler, D., Miller, W.: Aligning multiple genomic sequences with the threaded blockset aligner. Genome Research **14**(4), 708–715 (2004)

[11] Kuhn, R.M., Haussler, D., Kent, W.J.: The UCSC genome browser and associated tools. Briefings in Bioinformatics **14**(2), 144–161 (2013)

[12] Li, H.: Minimap2: pairwise alignment for nucleotide sequences. Bioinformatics **34**(18), 3094–3100 (2018)

[13] Hickey, G., Paten, B., Earl, D., Zerbino, D., Haussler, D.: HAL: a hierarchical format for storing and analyzing multiple genome alignments. Bioinformatics **29**(10), 1341–1342 (2013)

[14] Jones, D.C.: Coitrees: Cache Oblivious Interval Trees. https://github.com/dcjones/coitrees. GitHub repository (2020)

[15] Myers, G., Durbin, R., Zhou, C.: FastGA: fast genome alignment. Bioinformatics Advances **5**(1), 238 (2025)

[16] Kaushan, H., Marco-Sola, S., Garrison, E., Prins, P., Guarracino, A.: Adaptive tracepoints for pangenome alignment compression. bioRxiv (2026). https://doi.org/10.64898/2026.02.16.706236

[17] Deorowicz, S., Danek, A., Li, H.: AGC: compact representation of assembled genomes with fast queries and updates. Bioinformatics **39**(3), 097 (2023)

[18] Garrison, E., Sirén, J., Novak, A.M., *et al.*: Variation graph toolkit improves read mapping by representing genetic variation in the reference. Nature Biotechnology **36**(9), 875–879 (2018)

[19] Pangenome Team: sweepga: sweep-line based genomic alignment. https://github.com/pangenome/sweepga. GitHub repository (2024)

[20] Vaser, R., Sović, I., Nagarajan, N., Šikić, M.: Fast and accurate de novo genome assembly from long uncorrected reads. Genome Research **27**(5), 737–746 (2017)

[21] Erdős, P., Rényi, A.: On random graphs I. Publicationes Mathematicae Debrecen **6**, 290–297 (1959)

[22] Bolognini, D., Guarracino, A., Paleni, C., Dudley, T.S., Iacoviello, L., Raveane, A., Sudmant, P.H., Garrison, E., Soranzo, N.: Population-scalable genotyping from low-coverage sequencing data using pangenome graphs. bioRxiv (2026). https://doi.org/10.64898/2026.02.05.704023

[23] Nishikawa, J., Iizasa, H., Yoshiyama, H., Shimizu, N., *et al.*: Chromosomal fragile site breakage by EBV-encoded EBNA1 at clustered repeats. Nature **616**, 504–509 (2023)

[24] Sirén, J., Eskandar, P., *et al.*: Personalized pangenome references. Nature Methods **21**(11), 2017–2023 (2024)

Figures

Fig. 1: **IMPG: implicit pangenome graph concept and distributed construction. A.** Conceptual overview: pairwise alignments implicitly define a graph structure through transitive match relationships. IMPG indexes alignments using cache-oblivious interval trees and discovers sequences related through multi-hop alignment chains without materializing the graph. **B.** Partition-lace pipeline for distributed pangenome construction. Unlike chromosome-based or whole-sequence community approaches, alignment-based partitioning operates at interval granularity: individual contigs can be split across partitions at alignment boundaries, correctly handling translocations and rearrangements. Partitions are independently processed through graph construction or variant calling tools, then merged into unified GFA or VCF files.

Fig. 2: **EBV-associated tandem repeat polymorphism at 11q23 across 466 human haplotypes. A.** Distribution of repeat region lengths at the EBNA1-binding tandem repeat cluster (chr11q23) across all HPRCv2 haplotypes, colored by superpopulation. IMPG queried this ∼21 kb locus across 466 haplotypes in seconds on a single workstation. **B.** Repeat region length by population, showing continuous variation across ancestries that extends the range observed in four genomes by Nishikawa et al.

Methods

Here we provide algorithmic and implementation details not described in the main text.

Overview

IMPG is implemented in Rust and provides a command-line interface with nine primary operations: `index` (precompute alignment indices), `query` (direct and transitive coordinate projection), `partition` (divide pangenome into non-overlapping segments), `similarity` (pairwise sequence comparison), `refine` (optimize locus boundaries), `align` (generate sparsified alignment pairs), `graph` (end-to-end graph construction), `lace` (merge partitioned outputs), and `stats` (alignment index diagnostics). IMPG is freely available at https://github.com/pangenome/impg under the MIT license (version 0.4.1 used for all benchmarks).

Core algorithm: implicit pangenome graph

IMPG exploits a well-established property of pairwise alignments: they inherently define a graph structure through transitive match relationships [4, 8]. This principle underlies multiple alignment methods such as TBA/MULTIZ [10] and T-Coffee [25], which chain pairwise alignments transitively, and explicit graph builders like Cactus [9], which compute transitive closure from pairwise alignments. IMPG's contribution is to make this implicit structure efficiently queryable at runtime—discovering sequences connected through intermediate alignments by iteratively following alignment chains—without materializing the graph.

Index structure

The implementation utilizes cache-oblivious interval trees (coitrees) [14] that use a van Emde Boas recursive layout to represent tree structures implicitly using array indices rather than pointers. This layout achieves $O(\log_B n + k/B)$ cache misses for block size B, reducing memory overhead while maintaining efficient query performance. Sequence names are mapped to numeric identifiers through a deterministic sorting scheme, ensuring reproducible results. CIGAR strings are packed into compact 32-bit integers, with three most significant bits encoding the operation type and the remaining 29 bits encoding the operation length. Rather than loading all alignment data into memory, IMPG stores byte offsets to CIGAR data in the original files, enabling on-demand retrieval.

The index file structure uses a forest map—a lookup table that maps sequence IDs to their corresponding interval tree locations within the index file. Individual coitrees for each target sequence are stored sequentially, with byte offsets recorded in the forest map at the file's end. This enables lazy loading of specific target trees without parsing the entire index.

By default, each alignment A→B creates an implicit reverse mapping B→A, doubling the number of index entries but ensuring every haplotype can serve as a bridge for transitive queries in both directions. Users may opt for unidirectional indexing via `--unidirectional` for smaller indices at the cost of reduced transitive reach.

Alignment formats

Beyond PAF (both uncompressed and BGZF-compressed), IMPG supports the .1aln format [15] (ONE alignment format) and TPA (TracePoint Alignment) binary files [16]. Tracepoints [16] store sparse positional checkpoints that record sequence position deltas at complexity-bounded segment boundaries, achieving substantially more compact storage than text-based PAF: for the HPRCv2 dataset, TPA files are 5.5× smaller than BGZF-compressed PAF (986 MiB vs. 5.3 GiB for 7.1 M alignment records). CIGAR strings are reconstructed on demand via wavefront alignment (WFA) [26] within bounded bandwidth, guaranteeing exact reconstruction when the original sequences are available. For tracepoint-based formats, an approximate query mode bypasses CIGAR reconstruction for fast coordinate-level output, estimating projected boundaries from tracepoint segment coordinates without sequence I/O. Exact-mode queries reconstruct the full CIGAR via WFA [26] and produce base-exact coordinates. A comparison of interval boundaries between the two modes is provided in Supplementary Table X.

Multi-file index architecture

For large distributed datasets, IMPG supports per-file indexing with a unified sequence namespace. Each alignment file has its own .impg index and a serializable cache that stores the unified sequence index together with ID translation tables. Staleness detection based on file size and modification time ensures cache consistency. The indexing mode is selected automatically (≥100 files triggers per-file mode) or controlled via `--index-mode`. Critically, per-file indexing enables incremental indexing: new files require indexing while existing indices are reused without rebuilding.

Query algorithms

Basic overlap queries identify all alignments whose target intervals intersect a query range with $O(\log n + k)$ complexity. Query results include projected coordinates computed by traversing CIGAR operations to map target ranges to query coordinates.

Transitive queries iteratively follow alignment chains to discover sequences connected through intermediate alignments. The Erdős–Rényi random graph model [21] provides a heuristic guideline: when pairs are connected with probability $P > (1+\epsilon)\frac{\ln N}{N}$, the resulting random graph is connected with high probability using $O(N \log N)$ rather than $O(N^2)$ edges. Because genomic

alignments are not random—they depend on sequence similarity and population structure—this threshold serves as a practical lower bound rather than a formal guarantee. The default traversal depth is two hops (sufficient for reference-bridged datasets) and can be increased via `--max-depth`. Two traversal strategies are implemented: breadth-first search (BFS) for optimal cache locality and depth-first search (DFS) for fewer overlapping results in dense regions.

Partitioning algorithm

The `partition` command iteratively divides the pangenome into non-overlapping partitions using sliding windows of transitive queries, maintaining masked regions (already partitioned) and missing regions (not yet partitioned). Crucially, the algorithm operates at aligned-interval granularity: a single contig can be split across multiple partitions at alignment boundaries, with each interval assigned to the partition containing its homologous context. The sliding window mechanism scans reference coordinates, collecting transitively connected intervals into candidate partitions, and advances past already-partitioned (masked) regions. Four selection strategies control partition characteristics: longest (minimizes partition count), total (greatest unpartitioned content), sample (maximizes sample representation), and haplotype (maximizes haplotype diversity).

Comparison with partitioning approaches

Three approaches to pangenome partitioning differ in the granularity at which they assign sequences. Chromosome-based partitioning assigns contigs by chromosome identity, assuming synteny; this fails for translocations, inter-chromosomal rearrangements, and fragmented contigs that cannot be cleanly assigned to a single chromosome. Community-based partitioning, as implemented in PGGB's Leiden clustering [4], operates on sequence-similarity networks at whole-sequence granularity: entire contigs are assigned to communities, so a contig with regions homologous to different genomic contexts is forced into one partition. PGGB's documentation notes that translocations may be conflated with syntenic regions under this approach, and sequence similarity serves as a proxy for homology rather than a direct measure. IMPG's alignment-based partitioning operates at the level of aligned intervals: contigs are split at alignment boundaries, and intervals from the same contig are routed to different partitions when they align to different genomic contexts. This handles translocations naturally without special-casing and does not require synteny assumptions or whole-sequence similarity networks. The three approaches reflect different design goals—chromosome-based partitioning prioritizes simplicity, community-based partitioning balances scalability with sequence-level grouping, and alignment-based partitioning maximizes granularity at the cost of requiring indexed pairwise alignments.

Similarity analysis

The `similarity` command computes pairwise similarity coefficients between sequences in specified genomic regions. Overlapping sequences are extracted and a multiple sequence alignment generated. Three metrics are supported: Jaccard ($J = \mathsf{A} \cap B / \mathsf{A} \cup B$), Cosine ($\cos = \mathsf{A} \cap B / \sqrt{\mathsf{A} \cdot \mathsf{B}}$), and Dice ($D = 2\mathsf{A} \cap B / (\mathsf{A} + \mathsf{B})$). The resulting matrices can be processed through integrated PCA or MDS implementations with adaptive polarization for genome-wide consistency.

Locus refinement

The `refine` command optimizes genomic loci to maximize the number of entities (sequences, samples, or haplotypes) spanning both boundaries. This is particularly valuable for structural variant analysis where loci should avoid terminating within large insertions or deletions. The algorithm explores asymmetric left and right boundary expansions with configurable extension steps.

Alignment generation

The `align` command generates sparsified pairwise alignment sets using connectivity targets derived from the Erdős–Rényi model. Four strategies are supported: exhaustive ($O(N^2)$), random subsampling, connectivity-probability (default: 0.99), and tree-based sampling combining MinHash nearest-neighbor and stranger-joining pair selection [19]. In joblist mode, the command emits alignment commands for cluster distribution; direct execution via the embedded FastGA aligner is supported for smaller datasets.

Graph building engines

IMPG provides three graph construction engines, selectable via `--engine`:

Seqwish engine (default for `graph`). The `graph` command provides end-to-end pangenome graph construction from FASTA sequences: FastGA alignment via sweepga [19], filtering (scaffold chaining, cardinality control, identity and overlap filtering), seqwish-based graph induction [8], and GFA sorting via path-guided stochastic gradient descent.

POA engine. For small regions ($\leq$10 kbp), SPOA-based [20] partial order alignment produces a variation graph in a single pass. This engine applies padded POA (described below) when boundary context is needed.

Recursive engine (default for `query` and `partition` GFA output). A divide-and-conquer strategy that constructs variation graphs from sets of homologous sequences through recursive partitioning, parallel POA, and sub-graph lacing. The algorithm proceeds as follows:

1. *Recursion decision.* If the longest sequence span is at most `poa_threshold` (default 1 kbp) or recursion depth reaches `max_depth` (default 10), the base case applies directly. When the number of sequences exceeds

`seqwish_threshold` (default 500), seqwish is used instead of POA to avoid $O(N \cdot L)$ memory consumption.

2. *Alignment and anchor selection.* An all-vs-all alignment is computed via sweepga with k-mer frequency scaled to the sequence count ($N \times 10$, ensuring seeds are retained among N nearly-identical sequences). The longest sequence serves as the anchor.
3. *Chunk partitioning.* The anchor is divided into overlapping windows of `chunk_size` (default 5 kbp) with `padding` (default 100 bp) overlap on each side. For each window, alignment records are used to project the anchor's coordinate range onto every other sequence via linear interpolation through alignment blocks, supporting both forward and reverse strand mappings. Sequences lacking alignment to the anchor are included in every chunk as a fallback.
4. *Parallel recursion.* Chunks are processed in parallel via rayon, each recursively invoking the engine.
5. *Sub-graph lacing.* The resulting sub-GFAs are unified into a single graph. Node IDs are translated into a global namespace. Path ranges—encoded in path names as `name:start-end`—are sorted, deduplicated, and overlap-trimmed at chunk boundaries by walking node sequences to identify the overlap region and splitting or removing nodes accordingly. Contiguous ranges are linked with new edges, and unused nodes are removed.

Padded POA. When the POA base case operates on chunk boundaries, alignment quality can degrade at the edges of subsequences. Padded POA addresses this by extending each sequence by a configurable number of flanking bases on each side before running SPOA. After alignment, an MSA is generated to identify the columns corresponding to padding: for each sequence, the algorithm walks MSA columns consuming non-gap characters until the left-padding count is exhausted, then similarly from the right. The core column range is defined as the intersection across all sequences. A fresh SPOA graph is then built from only the core (non-padding) subsequences, ensuring that boundary artifacts from the padding context do not propagate into the final graph. In the recursive engine, padding is handled at the chunk-partitioning level (overlapping windows) rather than at the POA level, and the lacing step trims the resulting overlaps.

Alignment statistics

The `stats` command reports diagnostics on alignment indices: the number and total length of query and target sequences, overlap counts (total, mean, and median per sequence), the five most-connected target sequences, and bridge genome coverage—the fraction of target sequences that also appear as queries, indicating bidirectional index completeness. A warning is emitted when bridge coverage falls below 50% of targets, suggesting the index may benefit from rebuilding without `--unidirectional`. The `--list-sequences` flag prints a tab-separated list of sequence names and lengths.

Output merging

The `lace` command combines partition outputs: GFA merging resolves node ID conflicts and maintains path continuity; VCF merging performs coordinate sorting and contig validation. Output compression is supported in gzip, bgzip, and zstd formats.

Input and output formats

Input formats: FASTA files and AGC archives [17] for sequences; PAF (uncompressed/BGZF), .1aln [15], and TPA [16] for alignments. Output formats: BED, BEDPE, PAF (intervals and alignments); GFA via seqwish, POA, or recursive engines; MAF (multiple alignment); FASTA with optional gap characters and alignment information.

Benchmarking

We benchmarked IMPG on 389,234,668 pairwise alignments in 466 TPA files (one per target haplotype, 23.27 GiB on disk) from 25,221 random haplotype pairs sampled from the Human Pangenome Reference Consortium Phase 2 pangenome (HPRCv2, 466 haplotypes: 464 HPRC haplotypes plus CHM13 and GRCh38 references). Index creation completed in 17.5 s (PAF, 627 MiB index) and 10.5 s (TPA, 577 MiB index) using 16 threads on an AMD Ryzen 7 PRO 7840U workstation with 57 GiB RAM. Bidirectional indexing produced 14.3 M entries from 7.1 M records (not exactly doubled because self-alignments are not duplicated).

For querying the 6 Mbp MHC region (CHM13#0#chr6:28000000-34000000): direct queries completed in 1.60 s (PAF) and 1.04 s (TPA approximate) with 40–88 MiB peak memory; transitive queries in 1.98 s (PAF) and 1.61 s (TPA approximate). A per-file TPA dataset (466 files, 23.3 GiB total) demonstrated 0.83 s direct queries with warm cache. The complete ready-to-query representation—per-file TPA alignments (23.3 GiB), bidirectional indexes (14.7 GiB), TPA auxiliary indexes (1.4 GiB), and AGC archive (3.1 GiB)—totals 42.5 GiB. This is the full representation of 466 human haplotypes: no decompression, graph construction, or additional preprocessing is required to begin querying. At approximately 93 MiB per haplotype, the per-haplotype cost suggests that datasets of thousands of haplotypes remain tractable on commodity storage.

EBV repeat locus analysis

We queried the EBNA1-binding tandem repeat cluster at chromosome 11q23, reported at GRCh38 chr11:114,604,212–114,625,620 by [23]. Since GRCh38 is included among the 466 assemblies in the dataset, we queried directly from GRCh38 coordinates using `impg query` in transitive mode with approximate coordinate projection, projecting through the CHM13 reference to reach all haplotypes. Repeat region lengths were computed from

the projected intervals as the sum of all interval spans per haplotype. To quantify repeat copy number, we extracted FASTA sequences for all projected intervals using `impg query` with AGC-backed sequence retrieval and counted occurrences of the 18-bp EBNA1 binding motif (GGCAATATTTCGGCGCGA) and its reverse complement in each haplotype's sequence. Population labels (superpopulation assignments) for HPRC samples were obtained from the HPRC metadata. Analysis scripts are available in the repository (`paper-nmeth/scripts/run_ebv_analysis.sh`).

Data Availability

IMPG is freely available at https://github.com/pangenome/impg under the MIT license (version 0.4.1 used for all benchmarks). Benchmark scripts are included in the repository. Input assemblies are from the HPRCv2 release (https://humanpangenome.org); the CHM13v2.0 reference is available from the T2T Consortium (https://github.com/marbl/CHM13). Pre-computed alignments, TPA files, and IMPG indexes for the HPRCv2 dataset are available at Zenodo (DOI: https://doi.org/10.5281/zenodo.XXXXXXX).

Code Availability

All source code is available at https://github.com/pangenome/impg under the MIT license. Scripts for reproducing all analyses presented in this paper are included in the repository.

Methods References

[25] Notredame, C., Higgins, D.G., Heringa, J.: T-Coffee: a novel method for fast and accurate multiple sequence alignment. Journal of Molecular Biology **302**(1), 205–217 (2000)

[26] Marco-Sola, S., Eizenga, J.M., Guarracino, A., Paten, B., Garrison, E., Moreto, M.: Optimal gap-affine alignment in O(s) space. Bioinformatics **39**(2), 074 (2023)

A. **Extended Data Figures**
B. **Supplementary Tables**
C. **Supplementary Figures**

1 Extended Data Figures

Fig. 1: **IMPG index architecture. A.** Cache-oblivious interval trees index pairwise alignments by target coordinates, enabling efficient queries without explicit graph construction. **B.** Index file structure with forest map for lazy loading of target-specific trees.

Fig. 2: **Transitive query reach from a non-reference haplotype.** Querying the MHC region from a non-reference haplotype. Direct query recovers only 2 haplotypes (9 intervals); transitive query discovers all 466 assemblies (7,223 intervals)—a 233× ratio in haplotype reach, demonstrating that the implicit index captures pangenome-wide connectivity.

Fig. 3: **Per-file TPA dataset improves direct query reach.** Because per-file alignments include random pairwise comparisons beyond the reference, non-reference haplotypes have direct connections to multiple assemblies. Querying the MHC region from HG00097#1 directly discovers 581 intervals across 54 haplotypes—a 27× increase over the 2 haplotypes reachable via reference-only alignments.

2 Supplementary Tables

Table 1: IMPG performance on HPRCv2 alignments (464 HPRC haplotypes + CHM13 + GRCh38, 466 total). Queries target the 6 Mbp MHC region. All benchmarks on a 16-core AMD Ryzen 7 PRO 7840U workstation with 57 GiB RAM running IMPG v0.4.1.

Metric	**PAF (BGZF)**	**TPA (single)**	**TPA (per-file)**
Alignment size	5.3 GiB	986 MiB	23.3 GiB
Index size	627 MiB†	577 MiB†	14.7 GiB‡
Alignment records	7.1 M	7.1 M	*
Indexed entries	14.3 M†		*
Index time (16 thr.)	17.5 s	10.5 s	*
Direct query	1.60 s	1.04 s	0.83 s
Transitive query	1.98 s	1.61 s	*
Peak memory (query)	40 MiB	88 MiB	*

†Bidirectional: each record indexed as both forward and reverse mapping.
‡Sum across 466 independent per-file indexes.
*Per-file metrics depend on individual file sizes; not directly comparable.